ESPECIFICACIÓN TÉCNICA

Dashboard de Indicadores Financieros

Propuesta Técnica — Prácticas Empresariales 2025

Autor: Estudiante en Prácticas — Ingeniería de Software
Revisado por: Desarrollador Senior
Duración: 6 meses | Sprints de 2 semanas

---

1. Objetivo del Proyecto

Desarrollar una aplicación web fullstack que centralice los indicadores financieros y operativos del banco, permitiendo consulta interactiva con filtros por fecha y sucursal, y exportación de reportes. La aplicación estará dockerizada y lista para despliegue en entorno interno.

---

2. Arquitectura del Sistema

2.1 Diagrama de capas

La aplicación sigue una arquitectura de tres capas separadas por responsabilidad:

- Capa de presentación (Frontend): React SPA servida con Vite.
- Capa de negocio (Backend/API): FastAPI con Python, expone endpoints REST.
- Capa de datos (Base de datos): PostgreSQL con vistas y queries optimizados.

Flujo de datos:

```
Browser  →  React (Axios)  →  FastAPI  →  PostgreSQL
                        ←  JSON/JWT  ←
```

2.2 Principios de diseño

- API RESTful con versionado (/api/v1/).
- Autenticación stateless con JWT (access token + refresh token).
- Separación estricta entre lógica de negocio y acceso a datos (Repository pattern).
- Variables de entorno con .env para configuración (sin credenciales hardcodeadas).
- Docker Compose para orquestar frontend, backend y base de datos.

---

3. Stack Tecnológico

Capa	Tecnología	Justificación
Frontend	`React 18 + Vite`	Ecosistema amplio, HMR rápido, compatible con Recharts y Tailwind.
UI/Estilos	`Tailwind CSS`	Utility-first, sin conflictos de especificidad, fácil de mantener.
Gráficas	`Recharts`	Basado en D3, componentes declarativos, responsivo por defecto.
HTTP Client	`Axios`	Interceptores para adjuntar JWT automáticamente en cada request.
Backend	`Python + FastAPI`	Alto rendimiento, documentación automática con Swagger UI, tipado con Pydantic.
ORM	`SQLAlchemy + Alembic`	Gestión de modelos y migraciones de base de datos versionadas.
Auth	`python-jose (JWT)`	Access token (15min) + Refresh token (7 días), almacenado en httpOnly cookie.
Base de datos	`PostgreSQL 15`	Estándar bancario, soporte robusto para queries complejos y vistas materializadas.
Exportación	`openpyxl / ReportLab`	Excel (.xlsx) y PDF generados en el backend y descargados como stream.
Contenedores	`Docker + Compose`	Entorno reproducible, independiente del SO del servidor.
Control de versiones	`Git + GitHub`	Branches por feature (git flow simplificado), PRs para revisión de código.
Diseño UI	`Figma`	Mockups validados antes de desarrollar, evita retrabajo.

---

4. Estructura del Repositorio

Se usará un monorepo con separación clara entre frontend y backend:

```
dashboard-banco/
├── frontend/
│   ├── src/
│   │   ├── components/     # Componentes reutilizables
│   │   ├── pages/          # Vistas principales
│   │   ├── services/       # Llamadas a la API (Axios)
│   │   ├── hooks/          # Custom hooks
│   │   └── utils/          # Helpers y formateadores
│   ├── Dockerfile
│   └── vite.config.js
```

```
├── backend/
│   ├── app/
│   │   ├── api/v1/         # Endpoints por módulo
│   │   ├── core/           # Config, JWT, seguridad
│   │   ├── models/         # Modelos SQLAlchemy
│   │   ├── schemas/        # Schemas Pydantic
│   │   ├── services/       # Lógica de negocio
│   │   └── repositories/  # Acceso a datos
│   ├── alembic/            # Migraciones
│   ├── tests/              # Pruebas unitarias
│   └── Dockerfile
├── docker-compose.yml
└── README.md
```

---

5. Endpoints Principales de la API

Método	Endpoint	Descripción
POST	/api/v1/auth/login	Autenticación, retorna access + refresh token.
POST	/api/v1/auth/refresh	Renueva el access token usando el refresh token.
GET	/api/v1/transacciones/resumen	Volumen y monto total de transacciones con filtros.
GET	/api/v1/transacciones/tendencia	Serie temporal mensual para gráfica de línea.
GET	/api/v1/creditos/cartera	Monto total, tasa de mora, aprobados vs rechazados.
GET	/api/v1/clientes/nuevos	Clientes nuevos por mes con comparativa YoY.
GET	/api/v1/sucursales/rendimiento	Comparativa de indicadores entre sucursales.
GET	/api/v1/reportes/excel	Genera y descarga reporte Excel de los datos filtrados.
GET	/api/v1/reportes/pdf	Genera y descarga reporte PDF.

Todos los endpoints excepto /auth/ requieren Bearer token en el header Authorization. Los filtros comunes son: fecha_inicio, fecha_fin, sucursal_id.

---

6. Modelo de Base de Datos (Simplificado)

Las entidades principales del sistema y sus relaciones:

```
usuarios          → autenticación y control de acceso
sucursales        → catálogo de sucursales del banco
clientes          → registro de clientes (id, fecha_apertura, sucursal_id)
cuentas           → cuentas bancarias (id, cliente_id, tipo, saldo, estado)
transacciones     → movimientos (id, cuenta_id, tipo, monto, fecha, sucursal_id)
creditos          → solicitudes (id, cliente_id, monto, estado, tasa, fecha)
productos         → catálogo de productos financieros
solicitudes_prod  → relación cliente-producto con fecha y estado
```

Se usarán vistas materializadas en PostgreSQL para los KPIs más consultados, reduciendo el tiempo de respuesta de los endpoints de resumen.

---

7. Estrategia de Autenticación

Se implementará un flujo JWT estándar con doble token:

- Access Token: vida útil de 15 minutos, enviado en el header Authorization: Bearer <token>.
- Refresh Token: vida útil de 7 días, almacenado en httpOnly cookie (no accesible desde JS).
- Axios interceptor en el frontend renueva automáticamente el access token cuando recibe un 401.
- Los roles de usuario (admin, viewer) controlarán qué indicadores puede ver cada perfil.

---

8. Plan de Sprints (2 semanas cada uno)

Sprint	Período	Tareas técnicas
1	Semanas 1–2	Levantamiento de requisitos, diseño Figma, definición de esquema BD, setup del repo y ambientes.
2	Semanas 3–4	Modelos SQLAlchemy, migraciones Alembic, datos seed con Faker, primeros endpoints GET.
3	Semanas 5–6	Autenticación JWT completa, endpoints de transacciones y créditos con filtros.
4	Semanas 7–8	Setup React + Tailwind, layout principal, componentes de tarjetas KPI, integración Axios.
5	Semanas 9–10	Gráficas Recharts (línea, barras, donut), filtros de fecha y sucursal funcionales.
6	Semanas 11–12	Vistas de créditos y clientes, exportación Excel/PDF, pruebas manuales end-to-end.
7	Semanas 13–14	Conexión a fuente de datos real, ajuste de queries, optimización con vistas materializadas.

8	Semanas 15–16	Pruebas unitarias (pytest), pruebas de usabilidad con usuarios internos, correcciones.
9	Semanas 17–18	Dockerización completa (Compose), despliegue en servidor interno, pruebas de carga básicas.
10	Semanas 19–20	Documentación técnica (README, Swagger), manual de usuario, preparación de demo final.
11–12	Semanas 21–24	Buffer para imprevistos, mejoras post-feedback, presentación final.

---

9. Consideraciones Técnicas y Riesgos

Riesgo	Mitigación
Acceso limitado a datos reales del banco	Usar datos sintéticos con Faker en las primeras fases; integrar datos reales en el sprint 7.
Queries lentos con grandes volúmenes	Implementar vistas materializadas y paginación desde el sprint 3.
Cambios en requisitos a mitad del proyecto	Mantener revisiones quincenales con el gestor de producto para validar avance.
Restricciones de seguridad del banco	Revisar políticas de IT antes del sprint 1 para ajustar la arquitectura si es necesario.

---

10. Soporte Esperado del Desarrollador Senior

Se solicita la colaboración del desarrollador senior en los siguientes puntos clave del proyecto:

- Revisión de la arquitectura y estructura del repositorio al finalizar el Sprint 1.
- Code review en los PRs de autenticación JWT y endpoints críticos (Sprint 3).
- Orientación sobre las convenciones y estándares técnicos del banco.
- Apoyo para la integración con los sistemas de datos internos en el Sprint 7.
- Revisión final antes del despliegue en el Sprint 9.

Se estima un total de 4 a 6 horas de mentoría distribuidas a lo largo del proyecto. No se requiere dedicación continua.

---

11. Criterios de Aceptación Técnica

- Todos los endpoints documentados en Swagger UI y funcionando correctamente.
- Cobertura de pruebas unitarias del backend >= 70%.
- Tiempo de respuesta de endpoints de resumen < 500ms con datos reales.
- Aplicación dockerizada funcionando con docker-compose up en un solo comando.
- Código versionado en Git con historial limpio y mensajes descriptivos.

- README completo con instrucciones de instalación, variables de entorno y arquitectura.

Estudiante en Prácticas — Ingeniería de Software